

Please expand the documentation in the following code.

```
def paginate(items, page_size, page_number):  
    start = page_number * page_size  
    end = start + page_size  
    return items[start:end]
```

```
def flatten(nested):  
    result = []  
    for item in nested:  
        if isinstance(item, list):  
            result.extend(flatten(item))  
        else:  
            result.append(item)  
    return result
```

Here's the same code with expanded documentation, including purpose, parameters, return values, behavior, and examples:

 Python



 Run

```
def paginate(items, page_size, page_number):  
    """  
    Return a slice of a sequence corresponding to a specific page.  
  
    This function divides a collection into fixed-size pages and returns  
    the items that belong to the requested page. Pages are zero-indexed,  
    meaning that page_number=0 returns the first page, page_number=1  
    returns the second page, and so on.  
  
    Parameters  
    -----  
    items : sequence  
        The collection of items to paginate. Any object supporting  
        slicing (such as a list, tuple, or string) can be used.  
  
    page_size : int  
        The number of items to include on each page. Must be a  
        non-negative integer.  
  
    page_number : int  
        The zero-based index of the page to retrieve.
```

Returns

sequence

A slice containing the items for the requested page.

If the requested page is beyond the end of the collection, an empty sequence of the same type is returned by slicing.

Notes

The function does not perform validation on the input arguments.

Negative values for `page_size` or `page_number` will follow Python's normal slicing behavior, which may produce unexpected results.

Examples

```
>>> paginate([1, 2, 3, 4, 5, 6], 2, 0)
[1, 2]
```

```
>>> paginate([1, 2, 3, 4, 5, 6], 2, 2)
[5, 6]
```

```
>>> paginate([1, 2, 3], 2, 5)
[]
"""
```

```
start = page_number * page_size
end = start + page_size
return items[start:end]
```

```
def flatten(nested):
```

```
    """
```

Recursively flatten a nested list structure into a single list.

This function traverses a list that may contain other lists at arbitrary levels of nesting and returns a new one-dimensional list containing all non-list elements in their original left-to-right order.

Parameters

`nested` : list

A list whose elements may be values or additional nested lists.

Returns

list

A flat list containing every non-list element found in the input structure.

Notes

Only instances of `list` are flattened recursively. Other iterable types such as tuples, sets, generators, or strings are treated as ordinary values and are added to the result unchanged.

The function creates a new list and does not modify the input data.

Examples

```
>>> flatten([1, [2, 3], [4, [5, 6]]])  
[1, 2, 3, 4, 5, 6]
```

```
>>> flatten([])  
[]
```

```
>>> flatten([1, [2], "abc", [3, [4]]])  
[1, 2, 'abc', 3, 4]  
"""
```

```
result = []  
for item in nested:  
    if isinstance(item, list):  
        result.extend(flatten(item))  
    else:  
        result.append(item)  
return result
```

